

Orchestration Patterns

การประสานงานหลาย Agent แบบ Hub-and-Spoke

Domain 1 · Task 1.2 — Claude Certified Architect

Multi-Agent Orchestration คืออะไร?

- คือ **สถาปัตยกรรม**ที่แบ่งงานใหญ่ให้ agent หลายตัวช่วยกันทำ โดยมี agent กลาง 1 ตัวเป็น**ผู้ประสานงาน**
- ขอบสอนรูปแบบเฉพาะชื่อ **hub-and-spoke** (ดุมล้อ-ซี่ล้อ)



กฎเหล็กของ Task นี้

การสื่อสารทุกอย่างต้องวิ่งผ่าน coordinator เท่านั้น
subagent ห้ามคุยกันเองโดยตรง — เด็ดขาด

สองบทบาทในระบบ

Coordinator (อับกลาง)

รับงานเข้ามา → แยกงานเป็นงานย่อย → เลือก subagent ที่เหมาะ
→ ส่ง context ให้ → รวบรวมผลลัพธ์ → จัดการ error → กำหนดเส้น
ทางข้อมูล

Subagent (ซีล่อ)

agent เฉพาะทางที่ทำงานย่อยแยกกัน

web search

วิเคราะห์เอกสาร

สังเคราะห์ข้อมูล

สร้างรายงาน

แต่ละตัวรับผิดชอบงานคนละส่วน

หลักการ Context Isolation

★ ห้ามพลาด

🚫 **subagent ไม่สืบทอด (inherit) ประวัติของ coordinator โดยอัตโนมัติ**

subagent เข้าไม่ถึง:

- system prompt ของ coordinator
- ข้อความรอบก่อน
- ผลลัพธ์ของ subagent ตัวอื่น
- shared memory ใด ๆ

🔑 Key Concept

แต่ละครั้งที่เรียก subagent คือการเรียกที่เป็นอิสระต่อกัน ไม่มี memory ค้างระหว่างรอบ

ทุกข้อมูลที่ subagent ต้องใช้ → coordinator ต้องใส่ลงใน prompt อย่างชัดเจน

= explicit context passing

4 หน้าที่หลักของ Coordinator (1-2)

1 Dynamic Subagent Selection

วิเคราะห์ความต้องการของ query แล้วเลือกเรียกเฉพาะ agent ที่จำเป็น
query ง่ายใช้ agent น้อยลง — ไม่ต้องยัดทุกงานเข้า pipeline เต็ม

2 Research Scope Partitioning

เวลาทำงานให้หลาย subagent ต้องแบ่งขอบเขตให้ชัด
กำหนดหัวข้อย่อย / ประเภทแหล่งข้อมูลคนละส่วน → ลดงานซ้ำซ้อน

4 หน้าหลักของ Coordinator (ต่อ 3-4)

3 Iterative Refinement Loops

ประเมินผลการสังเคราะห์ว่ามีช่องโหว่ตรงไหน แล้วเรียกสังเคราะห์ซ้ำจนครอบคลุมพอ
ไม่ใช่กระบวนการยิงครั้งเดียวจบ (not single-shot)

4 Centralised Communication Routing

การสื่อสารของ subagent ทุกตัวต้องวิ่งผ่าน coordinator
→ ได้ observability · จัดการ error สม่่าเสมอ · ควบคุมการไหลของข้อมูล

นี่คือเหตุผลว่าทำไม subagent ถึงห้ามคุยกันเอง 🗑️

Narrow Decomposition Failure

★ ออกสอบบ่อย

อาการ

ระบบ multi-agent สร้างรายงานที่**ขาดทั้งหมดหุ้** — ไม่ใช่ขาดความลึก แต่**ขาดความกว้าง**

ตัวอย่าง

โจทย์ "ผลกระทบของ AI ต่ออุตสาหกรรมสร้างสรรค์" แต่ coordinator แยกงานเป็นแค่หัวข้อ **ทัศนศิลป์**

→ ขาด ดนตรี · งานเขียน · ภาพยนตร์ ไปทั้งหมด

🔍 หลักวินิจฉัย

รายงานขาดทั้งหมด → **อย่าโทษ subagent** ให้ไปตรวจ **decomposition** ของ coordinator ก่อน

ขาดในเชิงขอบเขต (ไม่ใช่เชิงความลึก) → ต้นเหตุมักอยู่ที่ decomposition เกือบทุกครั้ง



Case Study: รายงานพลังงานหมุนเวียน

อาการ

coordinator มอบหมายแค่ "**ประสิทธิภาพแผงโซลาร์**" กับ "**การออกแบบกังหันลม**"

สาเหตุ

ได้งานวิจัยลึกในสองหัวข้อนั้น แต่**ไม่มีข้อมูล** geothermal · tidal · biomass · nuclear fusion เลย

✅ วิธีแก้

ไม่ใช่ค้นให้เก่งขึ้น **ไม่ใช่**สังเคราะห์ให้ดีขึ้น และ**ไม่ใช่**เพิ่ม subagent

→ ปรับ **decomposition** ของ coordinator ให้ครอบคลุมทุกด้านตั้งแต่ต้น



Exam Traps ที่ต้องระวัง

1. โทษ subagent ปลายน้ำ

เรื่อง coverage gap ทั้งที่ต้นเหตุคือ decomposition ของ coordinator แคบเกินไป

2. คิดว่า subagent แอร์ memory

หรือสืบทอดประวัติของ coordinator — จริง ๆ context แยกกันสิ้นเชิง

3. ให้ subagent คุยกันตรง ๆ

เพื่อเพิ่มประสิทธิภาพ — ทำลาย observability · error handling · การไหลของข้อมูล

4. เพิ่มจำนวน subagent

เพื่อแก้ decomposition — งานที่มอบหมายแคบยังคงแคบ ไม่ว่าจะเพิ่ม agent ที่ตัว



Practice Scenario (ลองคิดก่อนดูเฉลย)

โจทย์: ระบบวิจัย multi-agent สร้างรายงานพลังงานหมุนเวียนที่ครอบคลุมแค่ **solar** กับ **wind** ทั้งที่งานของแต่ละ subagent ละเอียด มีแหล่งอ้างอิงดี และการสังเคราะห์ก็ถูกต้อง — **ปัญหาที่แท้จริงอยู่ที่ไหน?**

- A. subagent ค้นข้อมูลตื่นเกินไป ต้องปรับ prompt ให้แต่ละตัวค้นให้ลึกขึ้น
- B. subagent ไม่ได้แชร์ memory กัน ควรให้ subagent สื่อสารกันโดยตรงเพื่อเติมช่องว่าง
- C. มี subagent น้อยเกินไป ควรเพิ่มจำนวน subagent เข้าไปอีก
- D. coordinator แยกหัวข้อออกเป็นแค่ solar และ wind ไม่เคยมอบหมาย geothermal, tidal, biomass หรือ fusion

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อดูเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก: Option D

coordinator แยกหัวข้อออกเป็นแค่ solar และ wind ไม่เคยมอบหมาย geothermal, tidal, biomass หรือ fusion เลย

→ ต้นเหตุอยู่ที่ decomposition ไม่ใช่คุณภาพงานปลายน้ำ

ทำไมข้ออื่นผิด

A และ C — แก้ผิดจุด เพราะงานปลายน้ำ (การค้น + สังเคราะห์) ดีอยู่แล้ว การค้นให้ลึกขึ้นหรือเพิ่ม subagent ไม่ได้เติมหมวดที่ coordinator ไม่เคยมอบหมาย

B — ผิดหลัก hub-and-spoke โดยตรง subagent ห้ามคุยกันเอง และต่อให้คุยกันได้ก็ไม่มีใครรู้ว่าขาดหมวดไหนอยู่ดี



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

ระบบวิจัย multi-agent สร้างรายงานเรื่อง "renewable energy technologies" แต่ครอบคลุมแค่ solar กับ wind power เท่านั้น แต่ละ subagent ทำงานหัวข้อที่ได้รับมอบหมายอย่างละเอียดและมีแหล่งอ้างอิงดี ส่วน web search subagent ก็ค้นผลลัพธ์ที่ตรงประเด็นทุก query ที่ได้รับ — **อะไรคือ root cause ที่เป็นไปได้มากที่สุด?**

- A. web search subagent ใช้ query ที่แคบเกินไป เลยพลาดผลลัพธ์ของพลังงานประเภทอื่น
- B. synthesis subagent ล้มเหลวในการหาช่องโหว่ของงานวิจัย และไม่ได้ร้องขอให้ครอบคลุมเพิ่ม
- C. coordinator แยกหัวข้อออกเป็นแค่ solar และ wind ไม่เคยมอบหมาย geothermal, tidal, biomass หรือ fusion ให้ subagent ตัวไหนเลย
- D. document analysis subagent เข้าไม่ถึงแหล่งข้อมูลที่ครอบคลุมพลังงานหมุนเวียนประเภทอื่น

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option C

coordinator เป็นผู้รับผิดชอบการ decompose งาน ถ้ามอบหมายแค่ solar กับ wind ก็ไม่มี agent ปลายน้ำตัวไหนครอบคลุมหมวดที่หายไป เมื่อผลลัพธ์ขาดความกว้าง (scope) ต้นเหตุคือ decomposition ของ coordinator เสมอ

ทำไมข้ออื่นผิด

A — web search subagent ค้นตรงตามที่ได้รับมอบหมายและคืนผลตรงประเด็นแล้ว ปัญหาอยู่ที่ "ถูกสั่งให้ค้นอะไร" ไม่ใช่ "ค้นอย่างไร"

B — synthesis agent ทำงานกับงานวิจัยที่ได้รับมาเท่านั้น สังเคราะห์หัวข้อที่ไม่เคยถูกวิจัยไม่ได้ การหาช่องโหว่ระหว่าง iterative refinement เป็นหน้าที่ของ coordinator

D — ความพร้อมของแหล่งข้อมูลไม่ใช่ประเด็น coordinator ไม่เคยสั่งให้ใครค้นหมวดอื่น ต่อให้เข้าถึงแหล่งได้สมบูรณ์ หัวข้อที่ไม่ถูกมอบหมายก็ยังขาดอยู่ดี



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

ข้อใดคือ **ประโยชน์** ของการให้การสื่อสารของ subagent **ทุกตัว** วึ่งผ่าน coordinator?

- A. ลดจำนวน API calls รวมที่ต้องใช้ในการทำงานให้เสร็จ
- B. ทำให้ subagent แชนจ์ memory และต่อยอดผลลัพธ์ของกันและกันได้โดยอัตโนมัติ
- C. ได้ observability, การจัดการ error ที่สม่ำเสมอ และการควบคุมการไหลของข้อมูล
- D. ทำให้ subagent รับแบบ parallel ได้โดยไม่มี coordination overhead

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option C

การสื่อสารแบบรวมศูนย์ผ่าน coordinator ให้ประโยชน์ 3 อย่างชัดเจน: observability (log และ monitor ทุกข้อความที่จุดเดียว), consistent error handling (coordinator ใช้ recovery policy แบบเดียวกันทั้งระบบ) และ controlled information flow (coordinator เป็นคนตัดสินใจว่า subagent แต่ละตัวจะได้ context อะไร)

ทำไมข้ออื่นผิด

- A — การวิ่งผ่าน coordinator อาจ "เพิ่ม" API calls ด้วยซ้ำเมื่อเทียบกับการสื่อสารตรง ประโยชน์คือการควบคุมและมองเห็นได้ ไม่ใช่ประสิทธิภาพ
- B — subagent ไม่ได้แชร์ memory กัน การวิ่งผ่าน coordinator หมายถึง coordinator ส่งข้อมูลให้อย่างชัดเจน ไม่มี memory sharing อัตโนมัติ
- D — parallel execution เกิดจากการเรียก Task tool หลายครั้งใน response เดียว ไม่ใช่จาก routing pattern การรวมศูนย์เป็นเรื่องการควบคุม ไม่ใช่ parallelism



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

นักพัฒนาคนหนึ่งเสนอให้ subagent สื่อสารกันเองโดยตรงเพื่อลด latency — ทำไมแนวทางนี้ถึงมีปัญหา?

- A. Claude API ไม่รองรับการสื่อสารตรงระหว่าง subagent
- B. มันทำลาย observability, consistent error handling และ controlled information flow
- C. subagent ไม่สามารถประมวลผลข้อความจาก subagent ตัวอื่นได้
- D. มันจะทำให้ subagent แต่ละตัวต้องเข้าถึง tool ของ subagent ตัวอื่นทั้งหมด

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

การสื่อสารตรงระหว่าง subagent ข้าม coordinator ไป จึงทำลายประโยชน์ 3 อย่างของ hub-and-spoke: observability (ข้อความไม่ถูก log ที่ส่วนกลางแล้ว), consistent error handling (ไม่มี recovery policy แบบเดียวกัน) และ controlled information flow (ไม่มีผู้เฝ้าประตูคอยตัดสินว่าข้อมูลไหลไปไหน)

ทำไมข้ออื่นผิด

A — ปัญหาเป็นเรื่องสถาปัตยกรรม ไม่ใช่ข้อจำกัดทางเทคนิคของ API รูปแบบนี้ผิดไม่ว่า API จะรองรับหรือไม่

C — subagent ประมวลผล text input ใด ๆ ก็ได้ ปัญหาไม่ใช่ความสามารถในการอ่านข้อความ แต่คือการสูญเสียการควบคุมแบบรวมศูนย์

D — tool access เป็นคนละเรื่องกับ communication routing การสื่อสารตรงไม่ได้บังคับให้ต้องแชร์ tool กับ



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

coordinator เรียก web search subagent สองครั้งสำหรับสองหัวข้อย่อยที่ต่างกัน การเรียกครั้งที่สองคืนผลลัพธ์ที่ **ขัดแย้ง** กับครั้งแรก — อะไรอธิบายเรื่องนี้ได้?

- A. web search tool มี caching bug ที่คืนผลลัพธ์เก่า (stale)
- B. subagent ไม่แชร์ memory ระหว่างการเรียก — แต่ละครั้งเป็นอิสระและไม่รู้เรื่องครั้งก่อนหน้า
- C. coordinator ส่งคำสั่งที่ขัดแย้งกันให้การเรียกสองครั้งนั้น
- D. subagent ตัวที่สองสืบทอด context เก่าจากการเรียกครั้งแรกมา

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

subagent isolation หมายความว่าแต่ละการเรียกเป็นอิสระต่อกันโดยสมบูรณ์ การเรียกครั้งที่สองไม่รู้เลยว่าครั้งแรกเจออะไร ผลลัพธ์ที่ขัดแย้งจึงเกิดขึ้นได้เพราะแต่ละครั้งค้นแยกกันและอาจเจอแหล่งข้อมูลคนละชุด

ทำไมข้ออื่นผิด

- A — คำอธิบายอยู่ที่ subagent isolation ไม่ใช่ tool caching แต่ละการเรียกเป็นอิสระโดยการออกแบบอยู่แล้ว
- C — แม้คำสั่งที่ขัดแย้งจะทำให้ผลต่างกันได้ แต่โจทย์ถามว่าจะอธิบายความขัดแย้ง — เหตุผลเชิงสถาปัตยกรรมคือ subagent isolation (ไม่มี shared memory)
- D — subagent ไม่สืบทอด context จากการเรียกครั้งก่อน คำตอบนี้ขัดกับหลัก isolation โดยตรง



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

ผลลัพธ์ของระบบ multi-agent ขาดทั้งหมวดหมู่ของหัวข้อกว้าง ๆ อย่างต่อเนื่อง นักพัฒนาเสนอเพิ่ม specialist subagent อีกสามตัวเพื่อเพิ่มความครอบคลุม — วิธีนี้จะแก้ปัญหาได้ไหม?

A. ได้ เพราะ subagent มากขึ้นหมายถึงครอบคลุมหัวข้อได้ครบขึ้น

B. ไม่ได้ — ถ้า decomposition ของ coordinator แคบเกินไป subagent ใหม่ก็จะได้รับงานที่แคบพอ ๆ กัน วิธีแก้คือปรับ decomposition logic ให้ดีขึ้น

C. ได้ ตรงใดที่ subagent ใหม่เข้าถึง data source ที่ต่างออกไป

D. ไม่ได้ — วิธีแก้คือให้ subagent สื่อสารกันโดยตรงเพื่อให้มันหาช่องโหว่เองได้

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

root cause คือการ decompose งานของ coordinator ถ้า coordinator สร้างหัวข้อย่อยที่แคบ subagent ที่เพิ่มมา ก็จะได้รับงานที่แคบเท่านั้น วิธีแก้คือปรับปรุงวิธีที่ coordinator แยกหัวข้อให้ครอบคลุมความกว้างครบทุกด้าน

ทำไมข้ออื่นผิด

A — subagent มากขึ้นไม่ช่วยถ้ามันได้รับงานที่แคบ ความครอบคลุมขึ้นกับ "สิ่งที่ถูกมอบหมาย" ไม่ใช่ "จำนวน agent"

C — การเข้าถึง data source ไม่เกี่ยวเลยเมื่อ coordinator ไม่เคยมอบหมายหมวดที่หายไป agent ค้นหาค้นหาที่ไม่เคยถูกสั่งให้ค้นไม่ได้

D — การสื่อสารตรงระหว่าง subagent ทำลาย hub-and-spoke และไม่ได้แก้ปัญหา decomposition การหาช่องโหว่เป็นหน้าที่ของ coordinator ระหว่าง iterative refinement